Data-Driven Test Automation

Proposed Approach & Roadmap

Client: Clarien Bank — Finacle Automation Suite
Prepared: 6 Oct 2026
Finalised Approach: Excel-driven data + Menu-driven execution

Table of Contents

1. Executive Summary
2. Background & Client Observations
3. Design Objectives
4. Approach Comparison
5. Finalised Approach: Excel + Menu-Driven Runner
6. Detailed Roadmap
7. Sample Workbook & Code Sketches
8. Benefits, Risks & Mitigation
9. Next Steps

1. Executive Summary

The Finacle automation suite is functioning and the core test flows are stable. The remaining friction is operational: every time a new flow or a new data variant is needed, someone must touch the script files. For a business user with no JSON or TypeScript knowledge, that is a blocker.

This document proposes a fully **data-driven, business-user-friendly** model in which:

- All test data lives in a single Excel workbook.
- A dedicated *FlowConfig* sheet decides which spec runs with which data row.
- A generic runner reads the workbook, transforms the current step into environment variables, and executes the Playwright spec.
- The business user never edits scripts, JSON, or `package.json`. They pick a flow from a menu or double-click a batch file.

After a one-time refactor of the runner and the specs, every new flow is added by editing Excel only.

2. Background & Client Observations

During the review session, the client confirmed that all current tests and flows are running correctly. However, the following operational concerns were raised:

- Switching between flows requires re-configuring the runner to read from the correct data file.
- With multiple data files, the runner may become confused about which data source to use.
- The current model is not friendly to a business user who does not understand JSON or script code.
- The ideal end state is **data-centric**: script files remain untouched; only data, the execution menu and `package.json` (if a shortcut is required) are changed.

In short, the requirement is to move from a script-centric model to a **data + flow-definition** model.

3. Design Objectives

Objective	Why it matters
Scripts stay intact	Once the framework is generic, `.spec.ts` files and `runOrdered.js` never change for a new flow or a new data set.
Single source of truth	One Excel workbook eliminates the confusion of multiple JSON data files and makes it obvious which data belongs to which step.
Business-user ready	The client can edit data and even compose new flows in a familiar Excel grid without touching code.
Easy execution	A menu-driven runner or double-clickable batch files remove the need to remember npm commands or flow names.
Maintainable & traceable	Every change is in one workbook, is version-controllable (Excel is a binary, but the model is stable) and each run logs which flow and step data were used.

4. Approach Comparison

Three candidate architectures were discussed. The final selection is highlighted below.

Approach	How it works	Best for	Verdict
A. Central JSON + Flow Config	One `common-data.json` holds all entities. A separate `flow-config.json` lists each step, the data key it needs, and any overrides.	Technical teams who prefer source control, diffing and JSON editing.	Rejected for this client: requires JSON knowledge.
B. Excel + FlowConfig sheet	A single `data/test-data.xlsx` contains data sheets (Common, Savings, Current, Payment, Loans, Collateral) plus a *FlowConfig* sheet that decides the test order and data row for each step.	Business users and test analysts who are comfortable with Excel.	**Selected**
C. Hybrid: master JSON + manifest	Keep the existing `common-data.json` but enrich `testOrder.json` with data keys. Generic runner resolves data from the JSON manifest.	Teams that want the simplest evolution from the current `testOrder.json` design.	Rejected for this client: still JSON.

5. Finalised Approach: Excel + Menu-Driven Runner

The chosen architecture moves *all* data and *all* flow declarations into Excel. The scripts become a generic engine that interprets the workbook.

5.1 High-level architecture



5.2 Component responsibilities

Component	Responsibility
`data/test-data.xlsx`	Holds all data. Includes a *FlowConfig* sheet and one data sheet per business domain (Savings, Current, Payment, Loans, Collateral, Common, etc.).
FlowConfig sheet	Lists every flow, step number, spec file to run, data sheet, row id, enabled flag, shared-state inputs/outputs, and any special flags.
`tests/helpers/dataManager.ts`	One helper used by every spec. It reads the current step data from environment variables (or a temporary JSON written by the runner) and returns it as a typed object.
`scripts/runOrdered.js`	Generic. Loads the selected flow from the *FlowConfig* sheet, resolves the data row for each step, writes the step data, runs the Playwright spec, and continues until the flow ends.
`scripts/runFlowMenu.js`	Interactive menu that reads the list of available flows from the *FlowConfig* sheet and lets the user pick one by number.
Batch files	Optional `run-flow6.bat`, `run-flow7.bat` style shortcuts for one-click execution.

5.3 Data flow

1. User selects a flow from the menu or double-clicks a batch file.
2. The generic runner opens `data/test-data.xlsx` and reads the *FlowConfig* sheet for that flow.
3. For each enabled step, the runner looks up the requested data sheet and row, then exposes the fields as a JSON block in a temporary file or as environment variables.
4. The Playwright spec reads the step data through the shared `dataManager.ts` helper.
5. Shared state (e.g. an account id created in step 1 and used in step 5) is written to a small JSON file and re-injected into later steps as configured in the *FlowConfig* sheet.

6. Detailed Roadmap

The roadmap below is staged so that the suite stays runnable after every phase. Phases can overlap where dependencies allow.

Phase 1 — Data inventory & workbook design (2–3 days)

Goal: Produce the one-page data design that every later phase follows.

- Catalog every data point currently used in `common-data.json`, `flow1.json`, `flow6.json`, `flow7.json`, etc.
- Group data by domain: CIF, Savings, Current, Payment, Term Deposit, Retail Loan, Collateral, Common.
- Design the *FlowConfig* sheet columns: `flowName`, `step`, `spec`, `dataSheet`, `dataRowId`, `enabled`, `sharedStateIn`, `sharedStateOut`, `notes`.
- Define shared-state rules: which step creates an id, which step consumes it, and which field to map it to.
- **Deliverable:** `data/test-data.xlsx` skeleton with empty, correctly named sheets.

Phase 2 — Excel loader & data helper (3–4 days)

Goal: Make reading Excel as easy as the current JSON loader.

- Install or use the existing `xlsx` package.
- Build `scripts/excelLoader.js`: a small, reusable module that opens the workbook and returns rows by sheet + id.
- Build `tests/helpers/dataManager.ts`: a spec-side helper that reads the step data from the environment and returns a typed object. All specs import this helper.
- Write unit-style smoke tests that prove each sheet can be loaded and parsed.
- **Deliverable:** Excel loader and data helper passing a few sample lookups.

Phase 3 — Generic runner (3–5 days)

Goal: Replace the flow-specific code in `runOrdered.js` with a flow-agnostic engine.

- Refactor `scripts/runOrdered.js` to:
    1. Accept a flow name as an argument.
    2. Read the *FlowConfig* sheet and filter rows for that flow.
    3. For each enabled step, resolve the data row and write a temporary step-data JSON file or set environment variables.
    4. Run the Playwright spec with `--workers=1`.
    5. Read and update shared state between steps as configured.
- Remove the hard-coded `getFlow6StepEnv` and `getFlow7StepEnv` functions; their logic moves into the Excel data rows.
- **Deliverable:** A single `runOrdered.js` that can run any flow described in Excel.

Phase 4 — Spec refactor (4–6 days)

Goal: Make each `.spec.ts` consume data through the shared helper.

- Pick one spec from each domain (e.g. savings creation, payment order, retail loan creation) and refactor it to use `dataManager.ts` instead of hard-coded values or environment variables.
- Roll the pattern out to every spec in the suite.
- Remove direct reads of `process.env.FLOW6_*` etc. where they exist.
- **Deliverable:** All specs generic; no hard-coded flow data.

Phase 5 — Migration of existing flows (3–4 days)

Goal: Move all current data into the workbook and re-create all flows.

- Copy values from `common-data.json` and any per-flow JSON files into the relevant Excel sheets.
- Create a *FlowConfig* row for every entry in `tests/config/testOrder.json`.
- Verify each existing flow still passes when invoked from the new runner.
- Keep the old JSON files as a backup until Phase 6 is complete, then remove them.
- **Deliverable:** All old flows running from Excel data.

Phase 6 — Execution wrappers (2 days)

Goal: Let the business user run a flow without touching the terminal.

- Build `scripts/runFlowMenu.js`: list all flows from the *FlowConfig* sheet, ask the user to choose, then call `runOrdered.js`.
- Add a generic npm script: `"test-menu": "node scripts/runFlowMenu.js"`.
- Create optional per-flow `.bat` files so the client can double-click a flow.
- Test on the client's machine or a representative Windows environment.
- **Deliverable:** Menu and batch shortcuts ready.

Phase 7 — Validation & handover (2–3 days)

Goal: Prove the target state is stable and train the client.

- Run every flow end-to-end from the menu.
- Document the Excel workbook conventions and the meaning of each *FlowConfig* column.
- Demonstrate adding a new flow by only editing Excel and the menu.
- Archive old JSON data files and update `.gitignore` if needed.
- **Deliverable:** Signed-off, data-driven test suite and a short user guide.

6.1 Estimated timeline

Phase	Duration	Owner	Exit criteria
1. Inventory & workbook design	2–3 days	Automation lead + Business analyst	Workbook skeleton approved
2. Excel loader & data helper	3–4 days	Automation lead	Sample lookups pass
3. Generic runner	3–5 days	Automation lead	Any flow can be triggered
4. Spec refactor	4–6 days	Automation lead	All specs use data helper
5. Migration of flows	3–4 days	Automation lead + QA	Old flows pass from Excel
6. Execution wrappers	2 days	Automation lead	Menu and batch files tested
7. Validation & handover	2–3 days	Automation lead + Client	Client can add a flow independently

> **Total estimated effort:** 19–31 days. Parallel work in Phases 2/3 can reduce the calendar time by ~20%.

7. Sample Workbook & Code Sketches

7.1 Workbook sheets

Sheet	Purpose
FlowConfig	List of flows, step order, spec path, data sheet/row, enabled flag.
Common	Credentials, default customer, default SOL / CCY.
Savings	Scheme, SOL, initial funding, dispatch mode, related-party values.
Current	Current account scheme, funding HTM rows, modification flags.
Payment	Debit account, BIC, bank code, branch code, charge option, amount, currency.
Loans	Loan amount, scheme, repayment account, collateral links.
TermDeposit	GL sub-head, period, amount, repayment account.
Collateral	Lodgement, linkage, unlinking data.

7.2 FlowConfig sheet example

flowName	step	spec	dataSheet	dataRowId	enabled	sharedStateOut	sharedStateIn
flow7	1	tests/Finacle/savingsaccount/savingsaccountcreation.spec.ts	Savings	flow7-create	TRUE	accountId	
flow7	2	tests/Finacle/savingsaccount/savingsaccountcreationverify.spec.ts	Savings	flow7-verify	TRUE		accountId
flow7	3	tests/Finacle/transfermaintainence/accountfundingsavingsaccount.spec.ts	Payment	flow7-fund-1	TRUE		accountId

7.3 Spec using data helper

```
// tests/Finacle/savingsaccount/savingsaccountcreation.spec.ts
import { test } from '@playwright/test';
import { getStepData } from '../../helpers/dataManager';

test('create savings account', async ({ page }) => {
  const data = getStepData();
  // data contains schemeCode, solId, ccy, dispatchMode, etc.
  // All values came from the Excel row selected by the runner.
});
```

7.4 Menu runner skeleton

```
// scripts/runFlowMenu.js
const { chromium } = require('playwright');
const readline = require('readline');
const { loadWorkbook, getFlowNames } = require('./excelLoader');

const wb = loadWorkbook('data/test-data.xlsx');
const flows = getFlowNames(wb);

console.log('Available flows:');
flows.forEach((f, i) => console.log(`${i + 1}. ${f}`));

const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
rl.question('Pick a flow: ', (answer) => {
  const idx = parseInt(answer, 10) - 1;
  if (flows[idx]) {
    require('child_process').execSync(`node scripts/runOrdered.js ${flows[idx]}`, { stdio: 'inherit' });
  } else {
    console.log('Invalid choice.');
  }
  rl.close();
});
```

8. Benefits, Risks & Mitigation

8.1 Benefits

- **No-code data changes:** Business users edit Excel, not JSON or TypeScript.
- **Single workbook:** Eliminates confusion about which data file to use.
- **Flow composition in one place:** *FlowConfig* is the only place that decides which test runs in what order.
- **Reusable specs:** The same `.spec.ts` can participate in many flows by simply changing the data row id.
- **One-click execution:** Menu and batch files make running a flow as easy as opening a document.

8.2 Risks & mitigation

Risk	Mitigation
Excel is a binary file, so Git diffs are not readable.	Keep the workbook small and version-controlled; use a CSV fallback for major data changes, or document every change in a changelog sheet.
Business user may rename a sheet or column, breaking the runner.	Protect the workbook with a validation script that runs before the suite and checks sheet and column names.
Shared-state logic can become complex.	Define a fixed `sharedStateIn` / `sharedStateOut` contract and add clear examples in the user guide.
Initial refactor touches every spec.	Phase the refactor domain-by-domain, and keep old JSON data available until the new model is signed off.

9. Next Steps

1. Confirm the workbook domain split (Common, Savings, Current, Payment, Loans, Term Deposit, Collateral).
2. Approve the *FlowConfig* sheet columns and shared-state rules.
3. Schedule the start of Phase 1 (data inventory) and identify the business user who will own the workbook.
4. Once approval is in place, implement Phase 1 and Phase 2 to validate the Excel loader before touching the full suite.

The implementation will begin only after your go-ahead.